

使用 ADK、MCP 和 Memory Bank 建構個人化代理

來源：<https://codelabs.developers.google.com/codelabs/christmas-card/instructions?hl=zh-tw>

步驟數：11

瞭解如何建構可感知情境的 AI 虛擬服務專員，透過 MCP 連線至外部工具，並使用 Vertex AI Memory Bank 記住使用者偏好設定。

目錄

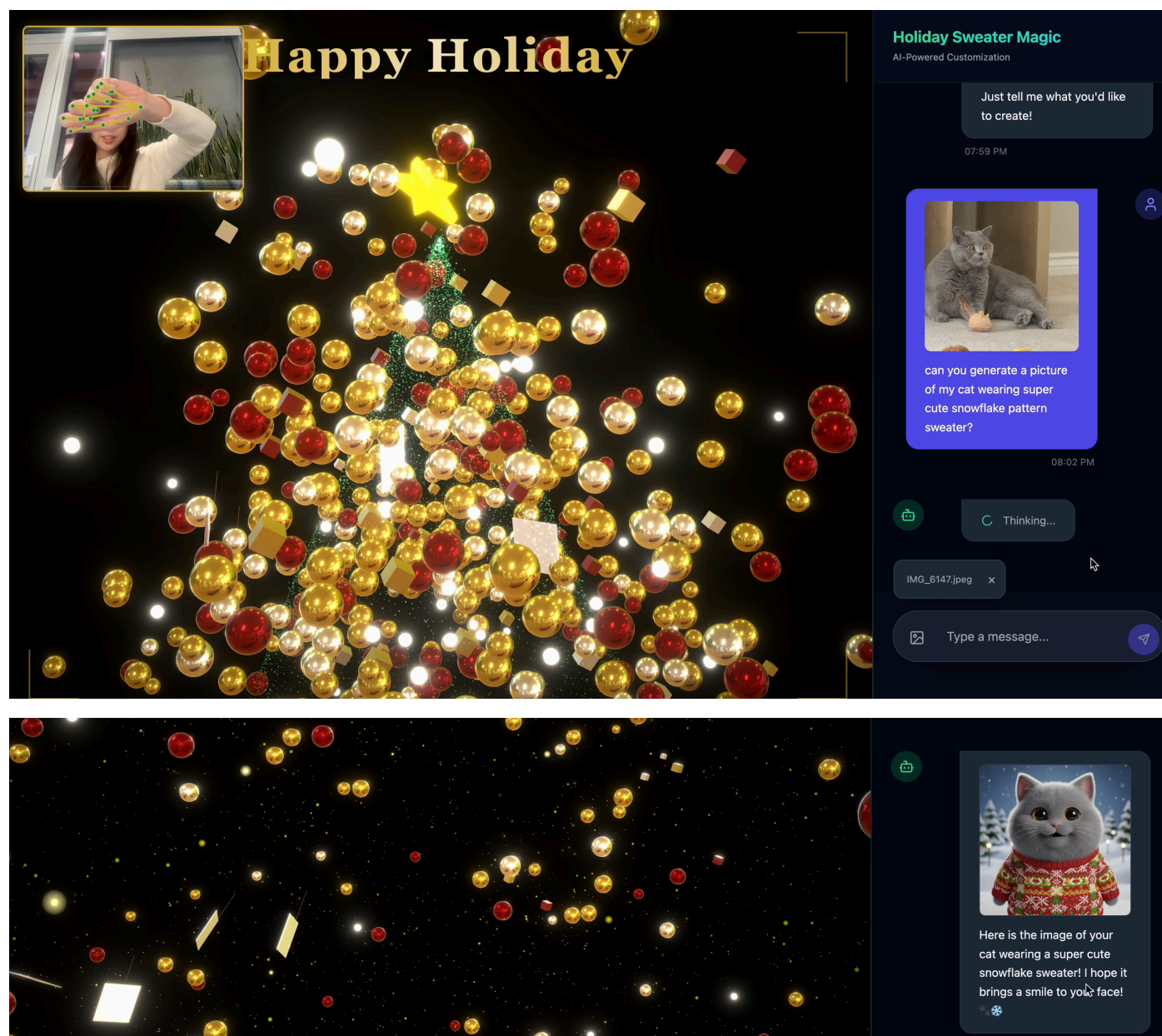
1. [簡介](#)
2. [設定](#)
3. [使用 MCP 強化](#)
4. [使用 Gemini CLI 測試 MCP 伺服器](#)
5. [以直覺式程式開發方式建立 ADK 代理](#)
6. [執行 Agent 網頁介面](#)
7. [將 ADK 連接至 UI](#)
8. [試用具有代理程式魔法的應用程式](#)
9. [Vertex AI Memory Bank](#)
10. [Memory Bank 實際應用](#)
11. [結論](#)

步驟 1 · 約 0 分鐘

1. 簡介

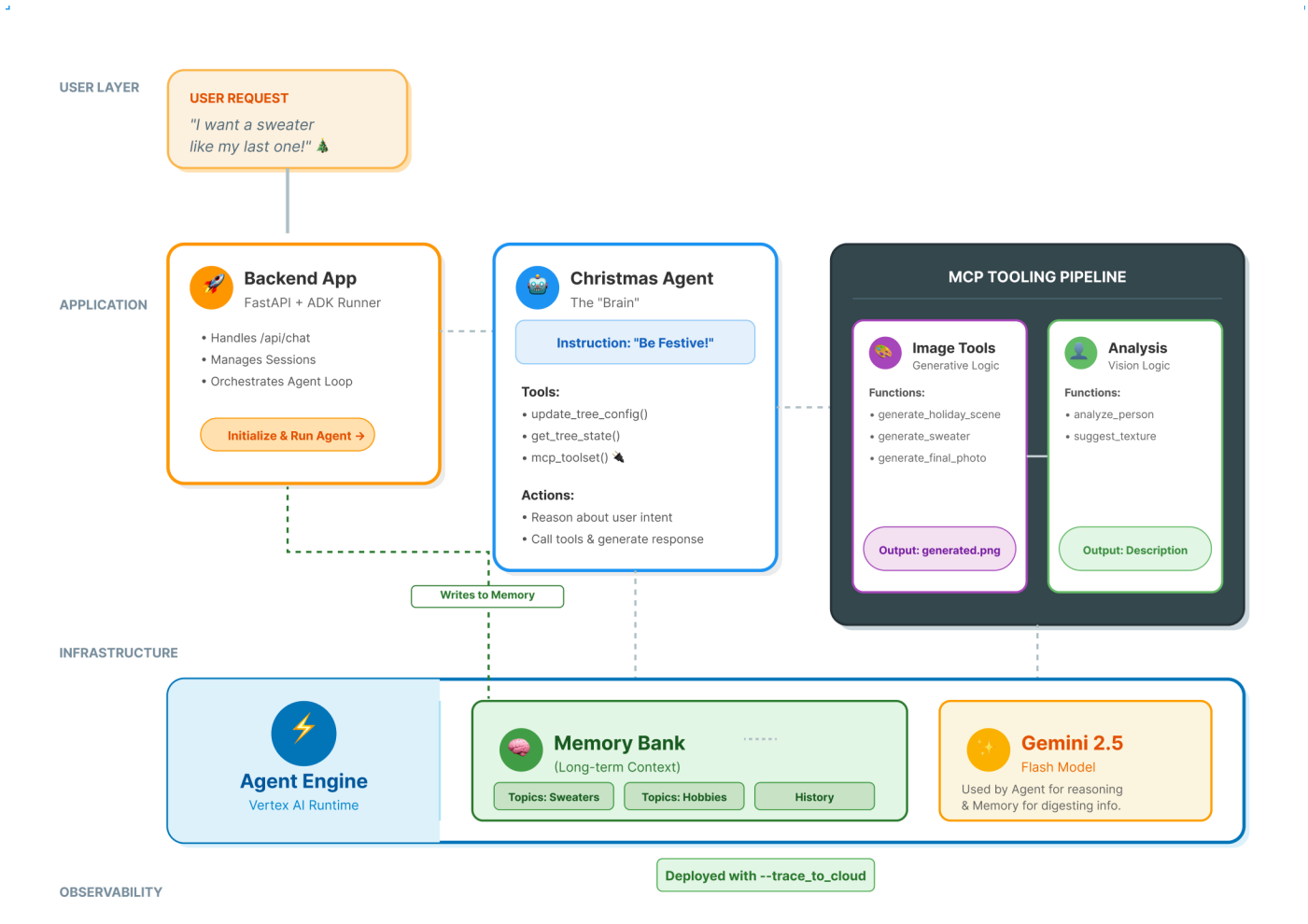
現代代理程式堆疊

打造可投入正式環境的 AI 代理，不只是建立大型語言模型 (LLM) 而已。LLM 提供推論功能，但強大的代理程式需要與外部世界互動、管理對話狀態，並長期記住使用者偏好。



在本研討會中，您將瞭解如何使用三項基礎技術，設計及建構完整的代理系統：

1. **連線 (MCP)**：讓代理存取本機工具和資料。
2. **自動化調度管理 (ADK)**：管理代理的推論迴圈和狀態。
3. **Memory Bank**：提供長期個人化脈絡。



核心概念

元件	函式
Model Context Protocol (MCP)	通用標準，可將 AI 模型連結至外部系統 (資料庫、檔案系統、API)，無須自訂整合。
Agent Development Kit (ADK)	這個架構可為代理提供執行階段環境，並管理事件迴圈、狀態轉換和工具執行作業。
工作階段服務	處理 短期記憶 。這項功能會保留當下的對話內容 (例如「使用者剛才問了什麼？」)，但工作階段結束時就會清除。
Vertex AI Memory Bank	處理 長期記憶 。這項功能會無限期保留使用者特定事實和偏好設定 (例如「使用者偏好 Python」)，讓服務專員在日後的互動中提供個人化服務。
Vertex AI Agent Engine	代管基礎架構服務，可大規模代管理程式邏輯和記憶體元件。

建構項目

為說明這些概念，您將建構**節慶設計助理**。這個代理程式能夠接收高階使用者要求，並自主協調本機 Python 工具，生成個人化程式碼和圖片。

您會經歷三個階段：

1. **工具層**：建立 MCP 伺服器，向 AI 公開本機 Python 函式。
 2. **代理層**：使用 ADK 建構代理，規劃及執行多步驟工作流程。
 3. **記憶體層**：整合 Memory Bank，讓代理程式學習並記住使用者的風格偏好。
-

步驟 2 · 約 0 分鐘

2. 設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

如要為 AI 代理提供支援，我們需要兩項要素：Google Cloud 專案做為基礎。

第一部分：啟用帳單帳戶

如要執行本程式碼研究室，您需要有具備部分抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

第二部分：開放式環境

1. 🖱️ 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 🖱️ 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

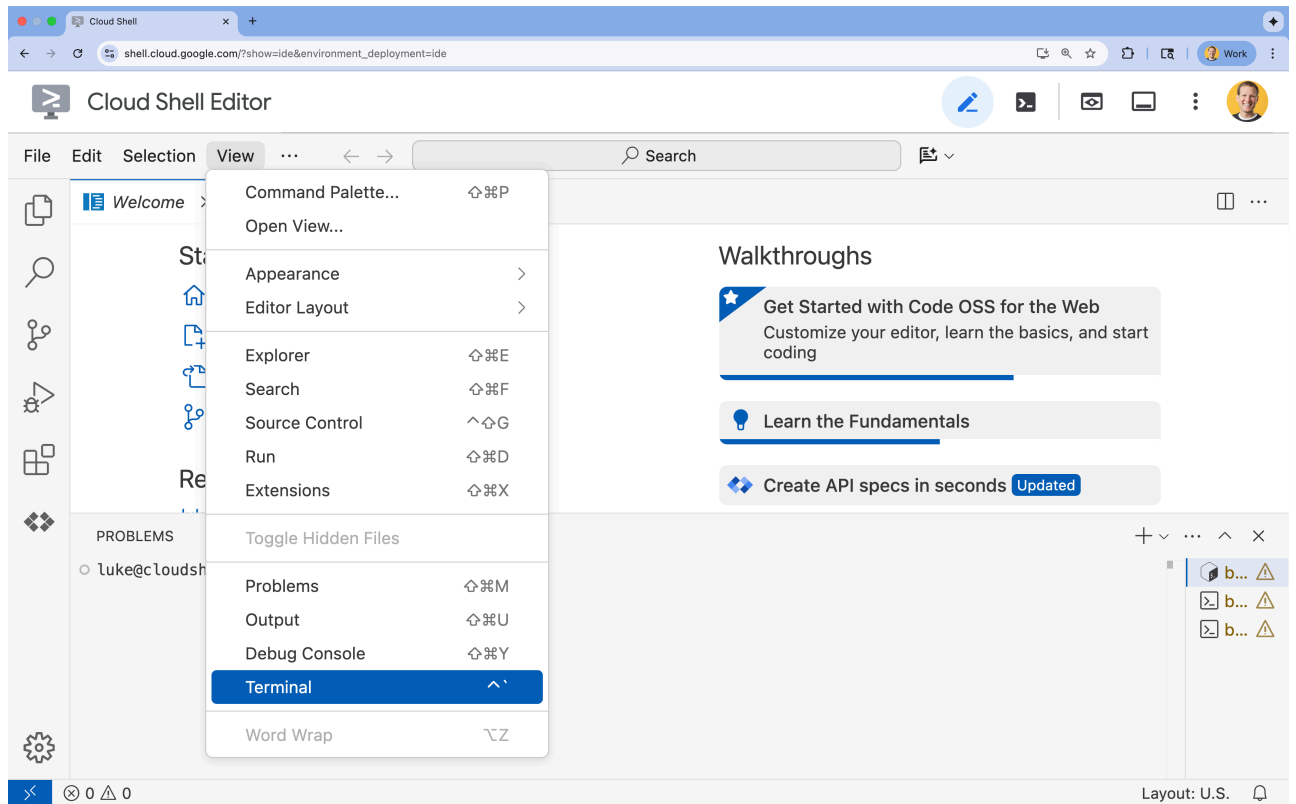
gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 🖱️ 如果畫面底部未顯示終端機，請開啟終端機：
 - 按一下「查看」
 - 按一下「終端機」



4. 🖱️ 在終端機中，使用下列指令驗證您是否已通過驗證，以及專案是否已設為您的專案 ID：

```
gcloud auth list
```

5. 🖱️ 從 GitHub 複製啟動程序專案：

```
git clone https://github.com/cuppibla/holiday_workshop
```

6. 🖱️ 從專案目錄執行設定指令碼。

```
cd ~/holiday_workshop
./init.sh
```

⚠️ **專案 ID 注意事項：**指令碼會建議隨機產生的預設專案 ID。您可以按下 **Enter** 鍵接受這項預設值。不過，如果您想建立特定新專案，可以在指令碼提示時輸入所需的專案 ID。

指令碼會自動處理其餘設定程序。

7. 🖱️ 設定所需的專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

第三部分：設定權限

1. 🖱️ 使用下列指令啟用必要的 API。這項作業可能需要幾分鐘才能完成。

```
gcloud services enable \  
  cloudresourcemanager.googleapis.com \  
  servicenetworking.googleapis.com \  
  run.googleapis.com \  
  aiplatform.googleapis.com \  
  compute.googleapis.com
```

2. 🖱️ 在終端機中執行下列指令，授予必要權限：

```
. ~/holiday_workshop/set_env.sh
```

請注意，系統會為您建立 `.env` 檔案。顯示專案資訊。

3. 使用 MCP 強化

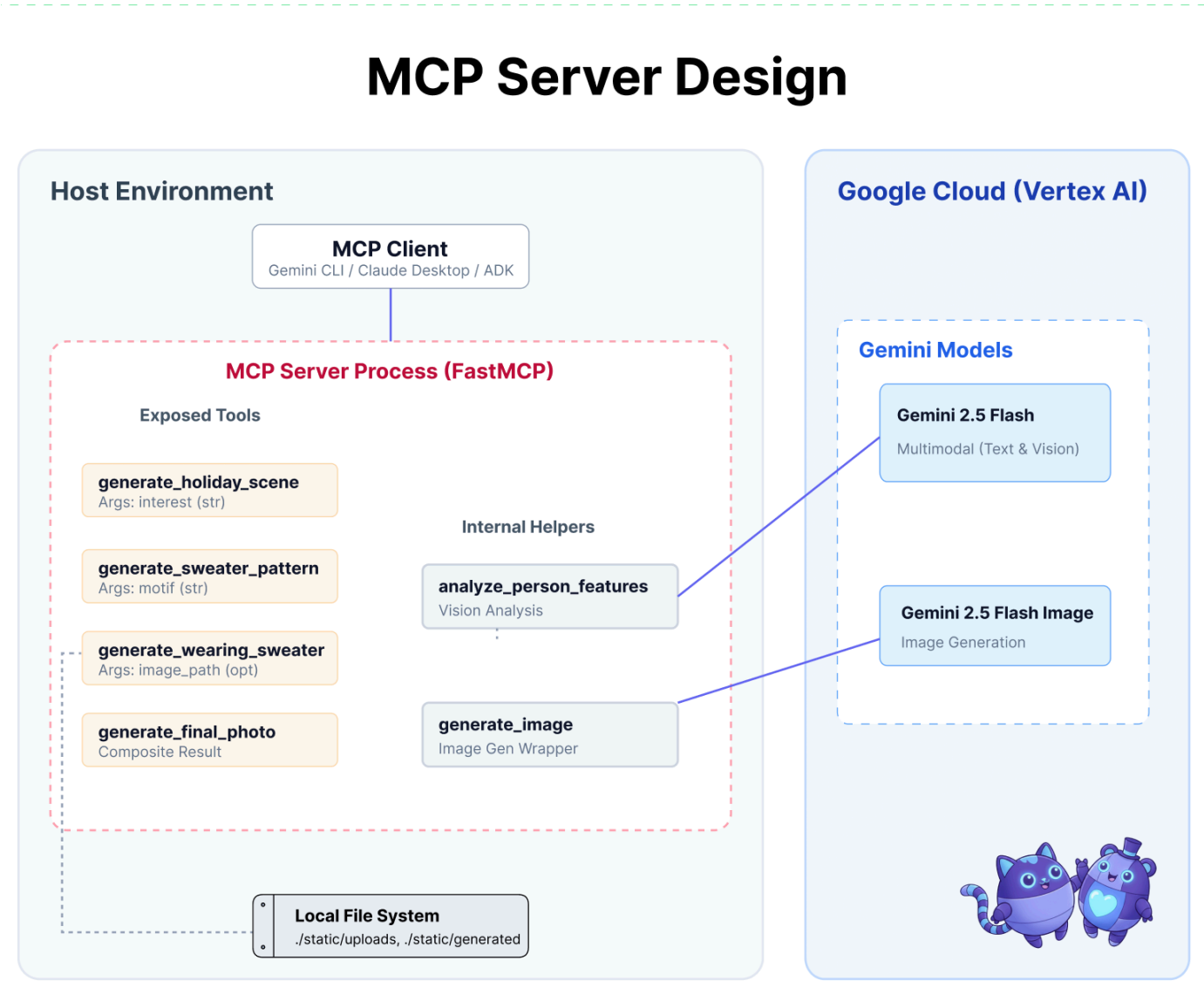
AI 的「USB-C」時刻

試想一下，每次購買新滑鼠時，都必須將滑鼠焊接到主機板上。這就是最近之前 AI 工具的狀態。開發人員必須編寫自訂「黏合程式碼」，才能將 LLM 連線至資料庫、檔案系統或 API。

輸入 **Model Context Protocol (MCP)**。不妨將 MCP 想像成 AI 應用程式的 **USB-C 連接埠**。這項技術提供標準化的方式，將 AI 模型連結至資料來源和工具。

只要為工具建構一次 MCP 伺服器，就能將其插入 Gemini CLI、IDE 或任何其他符合 MCP 規範的用戶端，完全不必變更任何程式碼。

建構項目



在本程式碼研究室中，您將建構「節慶設計助理」，這個助理具有以下功能：

1. 使用 **MCP**連線至本機環境 (Studio 工具)。
2. 使用 **Agent Development Kit (ADK)** 管理對話內容。
3. 記住您的偏好設定 (例如「我偏好 Python 程式碼」)，並在不同工作階段中使用 **Vertex AI Memory Bank**。

警告：執行程式碼範例前，請務必先驗證 Google Cloud CLI (`gcloud auth login`)。

建構伺服器邏輯

我們已準備好環境，但伺服器邏輯不完整。我們需要實作四項特定工具，這些工具最終會由 Agent 用來製作節慶卡片。

第一部分：開啟伺服器架構

我們將在 `01-MCP-Files-Testing/01-starter` 目錄中作業。

1. 在 Cloud Shell 終端機中，確認您位於正確的目錄：

```
cd ~/holiday_workshop/01-MCP-Files-Testing/01-starter/
```

2. 執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/holiday_workshop/01-MCP-Files-Testing/01-starter/mcp_server.py
```

您會發現樣板程式碼 (設定 MCP 伺服器、處理連線及初始化 Vertex AI 用戶端) 已完成。不過，目前這四項核心功能都是空白的預留位置。

第二部分：導入節慶場景產生器

首先，我們需要一個工具，可將使用者的興趣 (例如「鳥類」) 轉換為豐富詳細的提示，並針對圖片生成功能進行最佳化。

在 `generate_holiday_scene` 函式中找出 `#REPLACE_GENERATE_HOLIDAY_SCENE` 註解。

將整行程式碼替換為下列程式碼：

```
prompt = (
    f"""
    Create a cozy, high-fidelity 3D render of a winter holiday scene.
    The scene should be warm and inviting with soft cinematic lighting.

    Seamlessly integrate the following specific theme/interest into the
    holiday decor or landscape: {interest}.

    The style should be whimsical but detailed.
    Aspect Ratio: 16:9 Landscape.
    """
)
generate_image(prompt, "16:9", "static/generated_scene.png")
return "Done! Saved at generated_scene.png"
```

第三部分：實作最終相片結果

最後，我們希望確保光線和風格看起來寫實且充滿節慶氣氛。

找出註解 `#REPLACE_GENERATE_FINAL_PHOTO`。

將整行程式碼替換為下列程式碼，執行最終的樣式轉換和算繪作業：

```
prompt = (
    """
    Generate a photorealistic close-up shot of a rustic wooden fireplace mantle.

    Lighting: Warm, glowing ambient light from a fire below (out of frame).
    Background: Softly blurred (bokeh) pine garland and twinkling lights.

    Foreground Composition:
    1. A wooden picture frame containing the [attached selfie image].
       The face in the photo must be clearly visible.
    2. A folded holiday greeting card standing upright next to the frame.
       The front of the card displays the [attached holiday scene image] as a print.

    Ensure the perspective is grounded and realistic, as if taken with a 50mm lens.
    """
)
generate_image(prompt, "16:9", "static/generated_final_photo.png",
["static/generated_selfie.png", "static/generated_scene.png"])
return "Done! Saved at generated_final_photo.png"
```

環境設定

程式碼就位後，請務必安裝依附元件。我們將使用 `uv`，這是快速的 Python 套件和專案管理工具。

👉 在終端機中執行下列指令，將 FastMCP 新增為專案的依附元件：

```
cd ~/holiday_workshop/01-MCP-Files-Testing/01-starter/
uv add fastmcp
```

您會看到 `fastmcp>=2.13.3` 這個新的依附元件已新增至 `~/holiday_workshop/01-MCP-Files-Testing/01-starter/pyproject.toml` 檔案。

步驟 4 · 約 0 分鐘

4. 使用 Gemini CLI 測試 MCP 伺服器

伺服器程式碼已完成，接下來要如何測試？

通常測試後端伺服器時，需要建構前端 UI 或編寫複雜的 `curl` 要求。不過，我們可以使用 **Gemini CLI**。

什麼是 Gemini CLI？

Gemini CLI 是一種指令列工具，可讓您直接在終端機中與 Gemini 模型對話，而且還具備超能力：可自動探索及使用在本機 MCP 伺服器上執行的工具。

這項功能可隔離邏輯，因此對開發作業非常實用。您可以在不必擔心建構網頁介面或代理程式架構之前，先確認模型是否瞭解您的工具並正確呼叫。



連線並執行

我們會使用 `mcp add` 指令，要求 Gemini CLI 管理伺服器。

在終端機中執行下列指令：

```
gemini mcp add holidays uv run ~/holiday_workshop/01-MCP-Files-Testing/01-starter/mcp_server.py
```

- `add holidays`：我們為伺服器取了暱稱「holidays」。
- `uv run ...`：我們提供明確的指令，啟動剛才修改的 Python 伺服器。

一起大顯身手！

現在，請啟動即時通訊工作階段：

```
gemini
```


請嘗試下列提示，測試 Gemini 是否能「看到」新工具。請注意，您可能需要允許 Gemini CLI 使用節慶工具。

- 👉 使用者：

```
"I want to create a festive holiday photo. I like birds a lot."
```

- Gemini：

```
*Thinking...*
*Calling tool: generate_holiday_scene(interest='birds')*

Done! Saved at generated_scene.png
```

- 👉 使用者：

```
"Great! Now generate a knitting pattern for a sweater with reindeer on it."
```

- Gemini：

```
*Thinking...*
*Calling tool: generate_sweater_pattern(motif='reindeer')*

Done! Saved at generated_pattern.png
```

由於您使用了 MCP，AI 能夠準確判斷要呼叫哪個 Python 函式來滿足要求！

```
> generate a winter scene, I like birds a lot

✓ generate_holiday_scene (holidays MCP Server) {"interest":"winter scene with birds"}
Done! Saved at generated_scene.png
+ Done! Saved at generated_scene.png

Using: 2 GEMINI.md files | 3 MCP servers

> █ Type your message or @path/to/file

~/.../holiday-demo-2025 (main*)          no sandbox          auto
```

查看圖片

- 按下 **Control+C** 結束 Gemini CLI。
- 在資料夾中查看生成的圖片：`~/holiday_workshop/01-MCP-Files-Testing/01-starter/static`。

前往以下網址查看相片：

 generated_selfie.png U X

01-MCP-Files-Testing > 01-starter > static >  generated_selfie.png



結論與後續步驟

恭喜！您已成功建立可運作的 MCP 伺服器。您現在擁有一組實用的「AI 工具」，可生成模式、合成圖片及調整場景。

不過，您是否注意到上述測試中的某個現象？您必須主導整個程序。你必須先要求場景，然後要求模式，然後要求合併兩者。

雖然 Gemini 很聰明，但對於複雜的製作流程 (我們需要先生成圖案，才能將其放在毛衣上，並在圖像生成失敗時處理錯誤)，我們希望有更多控制權。我們希望系統能獨立規劃、自我評估，並管理節慶卡片的狀態，不必我們在每個步驟都提供協助。

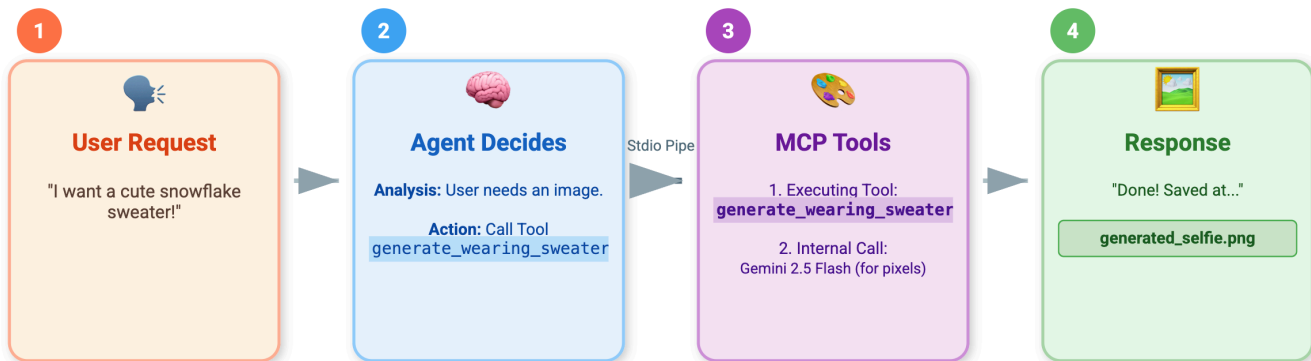
在下一節中，我們將整理這些混亂的廣告素材。我們將實作 **Agent Development Kit (ADK)**，建構結構化代理，將這些 MCP 工具整合到完美的製作管道中。

5. 以直覺式程式開發方式建立 ADK 代理

我們有一組可正常運作的工具 (MCP 伺服器)，但目前所有繁重的工作都由我們負責，也就是明確告知 Gemini 要呼叫哪個工具，以及何時呼叫。

在本節中，我們將建構 **AI 代理程式**，也就是能自主推論、規劃及執行多步驟工作的系統。為此，我們將使用 **Agent Development Kit (ADK)**。

Agent <-> MCP Walkthrough



什麼是代理程式？

如果 MCP 工具是「手」(負責執行工作)，則**代理**就是「大腦」。代理會使用 LLM 瞭解使用者的意圖(「幫我製作節慶卡片」)，然後將意圖分解成多個步驟(「首先需要場景，然後是圖案...」)，並決定要使用哪些工具來達成目標。

什麼是 ADK？

Agent Development Kit (ADK) 是 Google 提供的架構，可輕鬆建構這類代理。這項功能會處理複雜的「管道」作業，例如管理對話記錄、連結工具，以及在不同模型之間切換，讓您專注於應用程式的個性和邏輯。

情境式直覺程式開發

使用單一的大型提示詞生成程式碼是很常見的做法。不過，在建構複雜應用程式時，最好將 AI 視為合作夥伴，負責維護一段時間內的脈絡。

在編寫任何程式碼之前，我們會先使用 Gemini CLI 的**記憶體**功能設定舞台。

1. 準備環境

開啟終端機並前往入門目錄：

```
cd ~/holiday_workshop/02-Vibe-Coding-ADK-Agent/01-starter
```

啟動 Gemini CLI：

```
gemini
```

2. 管理脈絡和記憶

直覺式程式開發時，AI 必須知道 *自己是誰*，以及 *自己知道什麼*。Gemini CLI 可讓我們明確管理這項設定。

- `/memory show`：輸入這個指令，即可查看 AI 目前對專案和工作階段的瞭解程度。
- `/memory add`：使用這項功能注入基礎知識，讓 AI 在整個對話過程中都能記住。

首先，請定義程式碼合作夥伴的角色。在 Gemini CLI 內執行下列指令：

```
/memory add "You are an expert Python developer specialized in the Google Agent Development Kit (ADK). You write clean, modular code and prefer using the latest ADK patterns."
```

Gemini 現在瞭解自己的角色。這個脈絡會影響後續的所有回覆，確保程式碼符合 ADK 規定且品質優良。

3. 步驟 1：以直覺式程式開發編寫基本代理程式

我們不會嘗試一次生成整個系統，而是先從架構開始。我們想建立檔案結構和基本代理人個性。

在 Gemini CLI 中輸入下列提示詞：

```
Let's start by building the basic agent structure.

Please create a file structure for a `root_agent`.
1. Create `root_agent/__init__.py` that imports `agent`.
2. Create `root_agent/agent.py` by following exactly how this file is doing import and agent creation @~/holiday_workshop/02-Vibe-Coding-ADK-Agent/01-starter/agent_reference.py

In `agent.py`:
- Create an `Agent` named "root_agent" using the model "gemini-2.5-flash".
- The instruction string should define a "Holiday Magic Assistant".
- The personality should be enthusiastic (🌟🌟🌟) and prefer "cute, kawaii, cartoon" styles for any visual tasks.
```

Gemini 會生成檔案結構和初始 Python 程式碼。請先檢查變更是否正確，再套用/接受變更。

4. 步驟 2：新增 MCP 伺服器 (工具)

現在我們有了基本代理程式，需要為其「賦予雙手」。我們需要將代理程式連線至上一個實驗室中建構的 MCP 伺服器。

在 Gemini CLI 中輸入下列提示詞：

```
Now, let's give the agent access to tools. Update `agent.py` to include our local MCP server. By following exactly how this agent is connecting to mcp tool @~/holiday_workshop/02-Vibe-Coding-ADK-Agent/01-starter/agent_reference.py

In `agent.py`:
- Import `McpToolset` to define our STUDIO MCP server. as @~/holiday_workshop/02-Vibe-Coding-ADK-Agent/01-starter/agent_reference.py
- Connect to the python file located at `../mcp_server.py` relative to agent.py.
```

Gemini 現在會重構現有的 `agent.py`，加入工具定義和連線邏輯。

注意：如要檢查作業，或產生的程式碼無法如預期般運作，可以將檔案與以下位置的參考解決方案進行比較：`~/holiday_workshop/02-Vibe-Coding-ADK-Agent/solution`

步驟 6 · 約 0 分鐘

6. 執行 Agent 網頁介面

ADK 內建名為 `adk web` 的測試介面，這會啟動輕量型聊天 UI，方便我們立即與代理程式對話。

1. 如果 GeminiCLI 仍處於開啟狀態，請按 `control+C` 鍵關閉。現在在終端機(位於 `solution` 資料夾中，您可以前往 `starter`，在 `starter` 資料夾中執行 `uv run adk web` 來測試程式碼)，執行：

```
cd ~/holiday_workshop/02-Vibe-Coding-ADK-Agent/02-solution
uv run adk web --port 8000
```

2. Cloud Shell 會發出警示，指出服務正在通訊埠 8000 上執行。依序點選「Web Preview」(網頁預覽) -> 「Preview on port 8000」(透過以下通訊埠預覽：8000)。

測試代理

這時應該會看到即時通訊介面。讓我們看看 Agent 是否會遵循新指令，並正確存取 MCP 工具。

試試以下提示：

- 「你好，你是誰？」
 - (預期會收到熱情洋溢的回覆)。
- 「我想為節慶卡片找個背景，請製作雪村。」
 - (代理應呼叫 `generate_holiday_scene`。請注意，系統會自動套用系統指令中定義的「可愛/卡通」風格。
- 「Generate a sweater pattern with little pizza slices on it.」(生成有小披薩切片的毛衣圖案)，
 - (服務專員應撥打 `generate_sweater_pattern`)。

Event 2 of 4

<
>

✕

Event

Request

Response

Foot agent

generate_holiday_scene

generate_sweater_pattern

generate_wearing_sweater

generate_final_photo

```
modelVersion: "gemini-2.5-flash"
content:
  parts:
    - 0:
      functionCall:
        id: "adk-0d7b2ccc-0982-40e5-b639-bea66206eb49"
        args:
          interest: "cats"
          name: "generate_holiday_scene"
        thoughtSignature: "Cs8EAXLT2mSSC64LZ0VMMDDbXwL-
1z5mmeHxEP3BPzLpK0mrdFrJ368QNAvXx42NL0fRAcbrSovt2rDw95
DoRj20GH0eaatyOZ8FyI18Z1-0-
t5QHt15byGrK7no5HF1aUHYE3K2irc3GvHEvG15NH0vET0xp6aUESbjg
rONJGp8vm9s07A0qW0Ly5aDppAy8U00ddK1jpsKZF3GNPqww_pcF2tol
nTfnNltw-mM9yukHsvIqHpvTtAwWC6zaF0_w_ub5pf4xbFs26-
wgg8X44qyEV7Xwv03kqgKkUk7WjZ-
bd0M1Zew0AF3Y5KJ0CkxqCgY0qWhrzyN1SR9FkeTgCtGNPb2PT0u8C
nZ7_gZdAd5s32Naqpg6w3dd0DpsrYXh2b2CE1AC0dJp1XAF25pdxx6L
cwY8R1Qa3Dm3c1C212e910oFJpJg_R9kxboJCOsevyN1eynTePxeliWh
Pw0NkCpjoiIX1G6lbk01LitTYPFT_dk201tZrs3XrIhL7qgtPMM-
W0A4Agf0FEmdC8BuyBw8pTnh9luzDrLlghY9S8XdIo-0-
iwdHpcfg17HxhEOyWUBmEaayLlKer0FECqgI5Y3m-
MnAJENb00ksDsEt0w0P-J1XGnU8XV2tGUSJxvL-
t5y0vnm8II1u0Zht0a23E980T_ZtSF_x60F1tRdw6F1Hn4-
V00U74jnoB1a0pccp705H2R2px0dNYrvevY8t5cY25yl03NcueQ56fm
ySVpPmR80aisDf0NY1X2F58T0rdh110vMCR"
```

SESSION ID 824f9954-f686-4eb3-8a49-c4fce4da01681

USER ID user

Token Streaming

+ New Session

🗑

⬇

hi

hi

Hello there! 🎉 I'm your Holiday Magic Assistant, ready to spread some cheer! I can help you generate magical holiday images. Just tell me what you'd like to see! ✨

can you generate a winter holiday scene? I like cats!

⚡ generate_holiday_scene

✓ generate_holiday_scene

Absolutely! Here's a magical winter holiday scene featuring some adorable cats just for you! 🐾
(The image generated_scene.png will be displayed here.)

Type a Message...

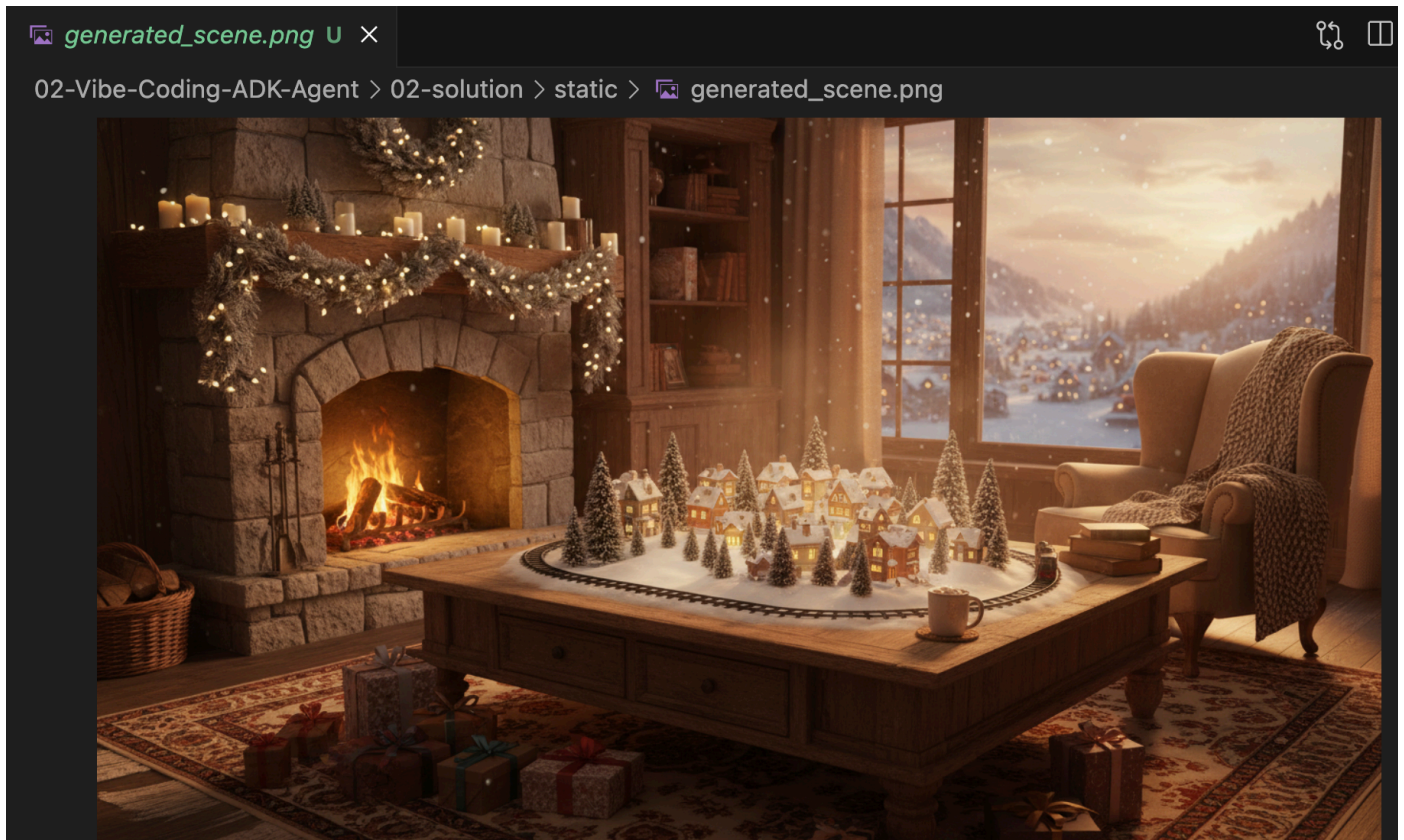
📎

⋮

🎤

📹

您可以在這裡查看生成的圖像：



完成測試後，請按下 `Control+C` 鍵結束。

結論與後續步驟

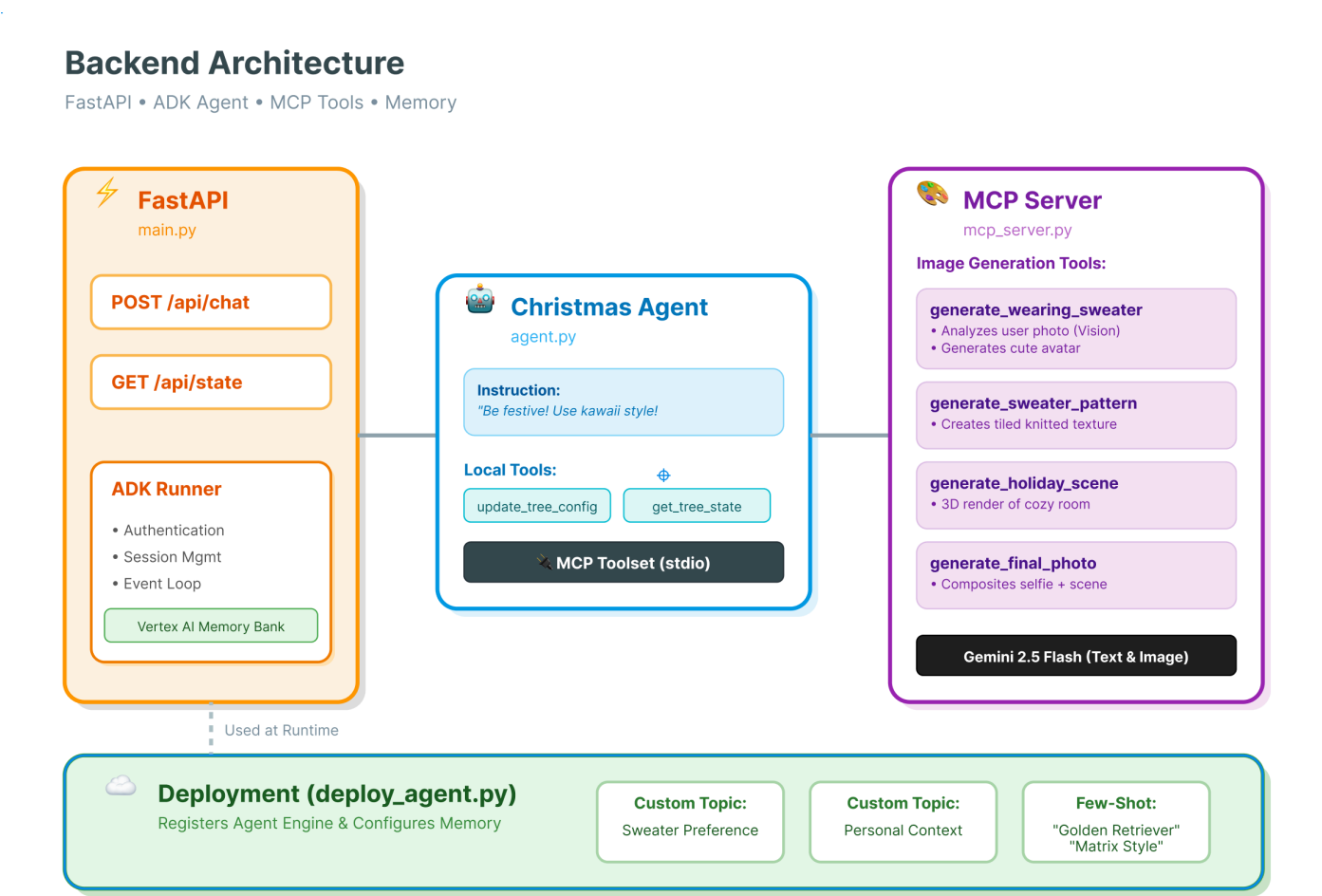
您現在已成功使用情境感知方法「Vibe-Coded」Google ADK 代理！

- 我們建立了情境：我們使用 `/memory add` 定義專家角色。
- 我們建構了 **Iteratively**：我們首先建立架構，然後新增工具連結。

內建的 ADK 網頁預覽功能很適合測試，但最終產品需要自訂的品牌體驗。在下一節中，我們會將這個 Agent 整合至自訂網頁前端。

如果在研討會期間遇到 `Unexpected response from metadata server: service account info is missing 'email' field`，表示您需要重新整理 Cloud Shell，然後重新執行 `gcloud config set project $(cat ~/project_id.txt) --quiet`，在繼續之前設定 `project id`

7. 將 ADK 連接至 UI



現在我們有了 Agent 定義，接下來需要運作執行。這時 **Runner** 和 **Session Service** 就派上用場了。

導入作業

1. 🖱️ 在指令中輸入下列內容：

```
cloudshell edit ~/holiday_workshop/03-Connect-ADK-MCP-UI/01-starter/backend/main.py
```

這會在編輯器中開啟 `~/holiday_workshop/03-Connect-ADK-MCP-UI/01-starter/backend/main.py`。

2. 將 `# TODO: Create Session Service` 替換為下列內容：

```
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
session_service = InMemorySessionService()
memory_service = InMemoryMemoryService()
```


工作階段服務是代理程式的「工作記憶」。

- **角色**：儲存當前對話的*即時記錄* (例如「我剛問了什麼？」)。
- **兩種口味**：
 - `InMemorySessionService`：快速、簡單，但會忘記。如果重新啟動伺服器，代理程式會忘記所有內容。非常適合測試。
 - `VertexAiSessionService`：功能完備，以資料庫為基礎。這項功能會將狀態儲存至 Google Cloud，因此對話可以在幾週後繼續，或在伺服器重新啟動後繼續。

3. 將 `# TODO: Initialize Runner` 替換為下列內容：

```
runner = Runner(  
    app_name="agents",  
    agent=christmas_agent,  
    session_service=session_service,  
    memory_service=memory_service,  
)
```

執行元件：如果代理是「大腦」，**執行元件**就是「心臟」。讓血液持續流動。Runner 負責：

- 將 Agent 繫結至使用者和工作階段。
- 執行事件迴圈。
- 管理狀態轉換。

4. 請查看第 158 行(`~/holiday_workshop/03-Connect-ADK-MCP-UI/01-starter/backend/main.py` 不需要採取任何行動)：您是否想知道應用程式如何取得最終回覆？以下是透過執行器啟用的事件迴圈：

```
async for event in runner.run_async(  
    user_id=user_id,  
    session_id=session_id,  
    new_message=content  
)
```

事件迴圈 ADK 代理不會只是「回覆」，他們會不斷重複思考，呼叫 `runner.run_async()` 時：

1. 使用者輸入進入迴圈。
2. 規劃：代理會考量可用的工具和目標。
3. 行動：視情況呼叫工具 (例如 `list_files`)。
4. 觀察：讀取工具輸出內容。
5. 反思：判斷是否需要更多資訊，或是否能回答問題。
6. 迴圈：重複執行步驟 2 到 5，直到滿意為止。
7. 回應：將最終答案提供給使用者。

深入探討：架構與部署

我們使用 **FastAPI** 提供這個代理程式。

- **為什麼選擇 FastAPI？**：代理程式通常會受到 I/O 限制 (等待 LLM)。FastAPI 的非同步特性可完美處理這項作業。
 - **無狀態**：請注意，我們的 API 端點是無狀態。我們不會儲存全域範圍內的變數。我們會依據 `session_id` 和 `SessionService` 重建每個要求的狀態。也就是說，您可以將這個應用程式部署至 **Cloud Run** (無伺服器)，並將規模縮減至零！
-

步驟 8 · 約 0 分鐘

8. 試用具有代理程式魔法的應用程式

1. 🖱️ 在指令中輸入下列內容：

```
cd ~/holiday_workshop/03-Connect-ADK-MCP-UI/01-starter
./start_app.sh
```

這會在編輯器中開啟 `~/holiday_workshop/03-Connect-ADK-MCP-UI/01-starter/backend/main.py`。

2. 您會看到如下結果：

```
> frontend@0.0.0 dev
> vite --host

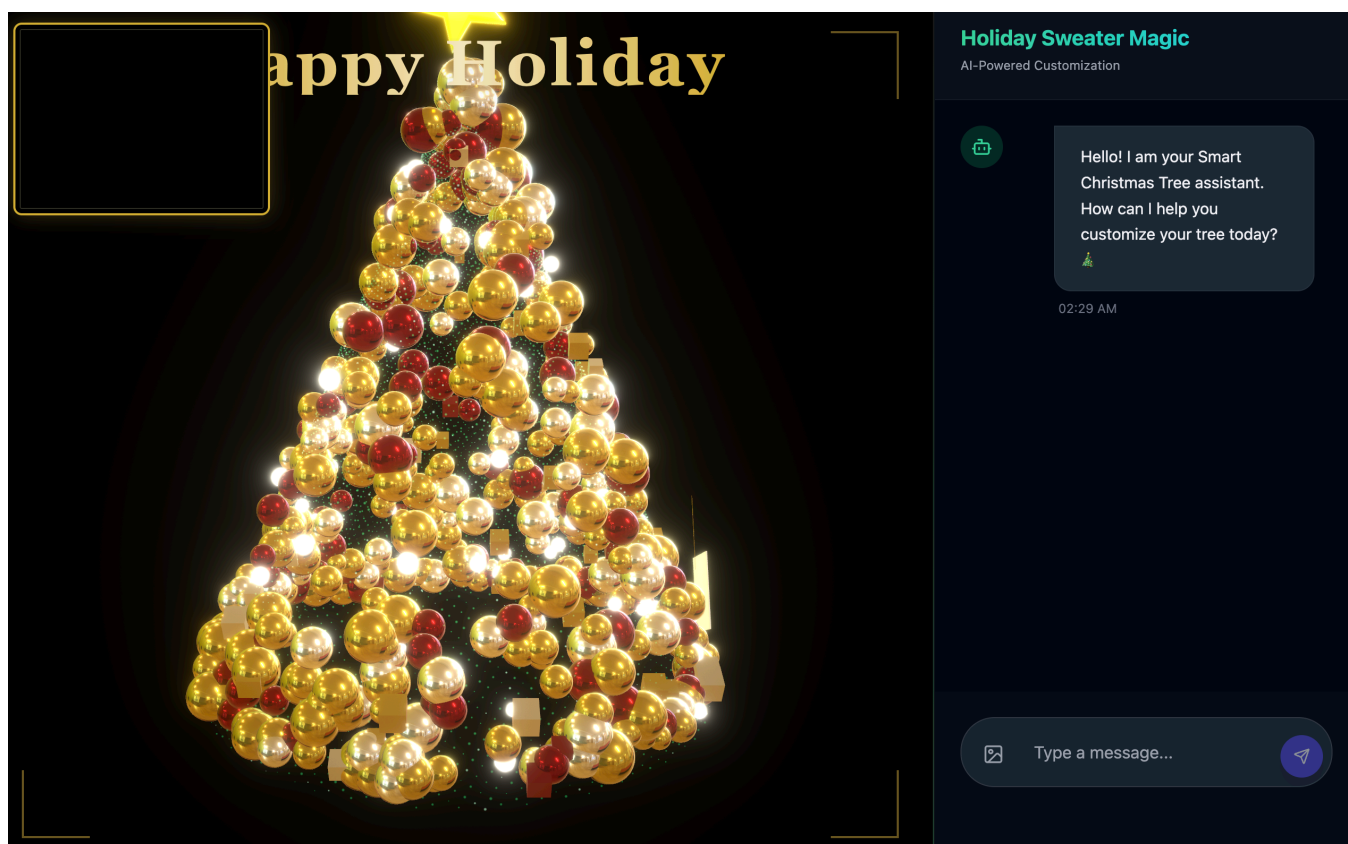
VITE v7.2.7 ready in 313 ms

➔ Local:   http://localhost:5173/
➔ Network: http://10.88.0.3:5173/
WARNING:agent:GOOGLE_API_KEY not found in environment variables. Agent may fail to initialize.
INFO:      Started server process [13267]
INFO:      Waiting for application startup.
```

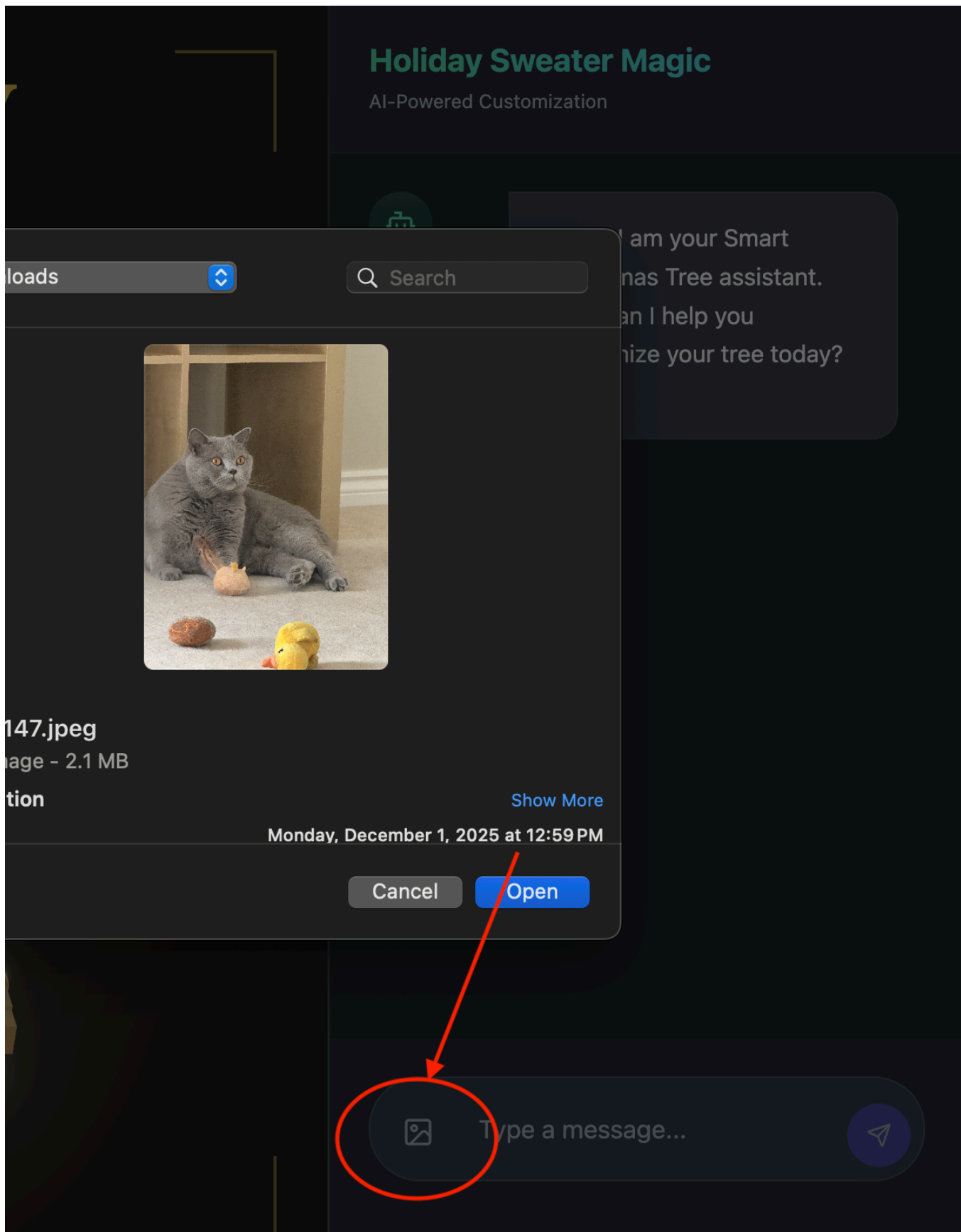
🖱️ 請務必點選 `http://localhost:5173/`，或開啟新視窗並輸入 `http://localhost:5173/`。

請注意，這「不是」後端伺服器的連結：`Uvicorn running on http://0.0.0.0:8000`

3. 然後您會看到含有對話介面的網站：



4. 上傳一張圖片進行測試(可以是您自己或寵物的相片)



5. 🖱️ 接著詢問

Can you generate a picture my cat wearing snowflake pattern sweater?

生成的圖像會顯示在這裡：

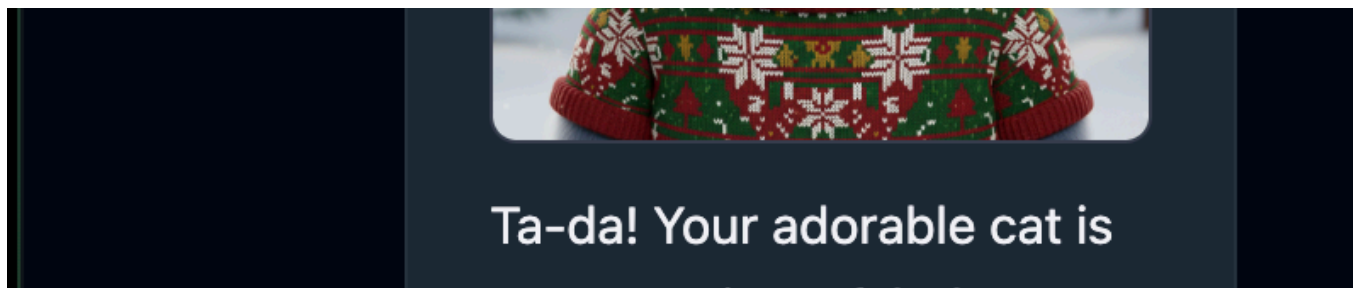
o



Can you generate a picture
my cat wearing snowflake
pattern sweater?

02:30 AM

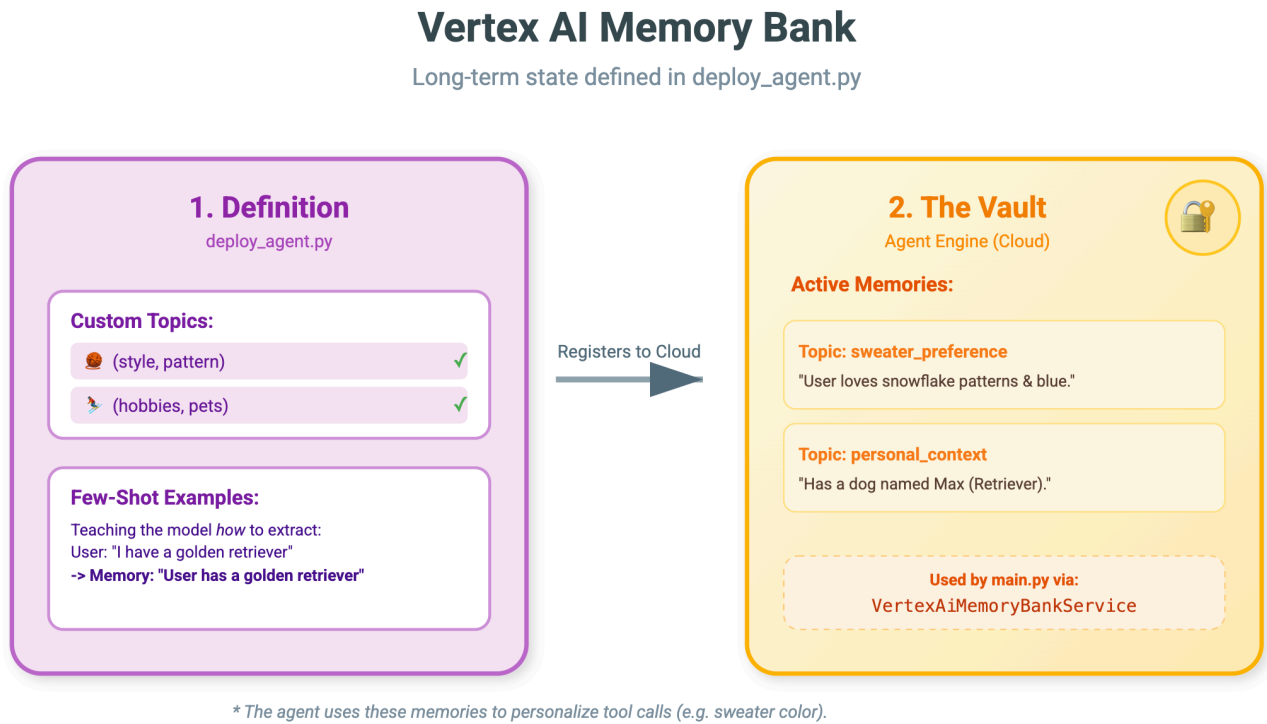




6. 🖱️ 測試完成後，請在終端機中按下 `control+c` 結束程序。

如果發現運作不如預期，可以前往 `~/holiday_workshop/03-Connect-ADK-MCP-UI/02-solution` 執行 `./start_app.sh`，然後重複上述步驟。

9. Vertex AI Memory Bank



短期記憶與長期記憶

- **短期脈絡**：「我剛說了什麼？」(工作階段記錄)。關閉即時通訊視窗後，這些資訊就會遺失。
- **長期記憶**：「我最喜歡的程式設計語言是什麼？」(使用者偏好設定)。這項設定應永久保留。

Vertex AI Memory Bank 提供這項長期儲存功能。代理程式可藉此儲存及擷取使用者的個人化資訊。

工作階段與 Memory Bank

- **工作階段** (`VertexAiSessionService`)：這是**記錄**。這個檔案會依時間順序儲存每則訊息、工具呼叫和事件 (`AppendEvent` 、 `ListEvents`) 的原始序列，提供**實際發生的情況**。
- **Memory Bank** (`VertexAiMemoryBankService`)：這是**知識**。儲存合成的長期事實 (`GenerateMemories` 、 `RetrieveMemories`)，範圍限定於特定 `user_id`，確保隱私權和隔離。

1. 在指令中輸入下列內容：

```
cloudshell edit ~/holiday_workshop/04-Adding-Memory-Bank/01-starter/backend/main.py
```

這會在編輯器中開啟 `~/holiday_workshop/04-Adding-Memory-Bank/01-starter/backend/main.py`。

2. 找出 `# TODO: Create Vertex AI Session Service & Memory Bank Service`，然後將**整行**替換為下列程式碼：

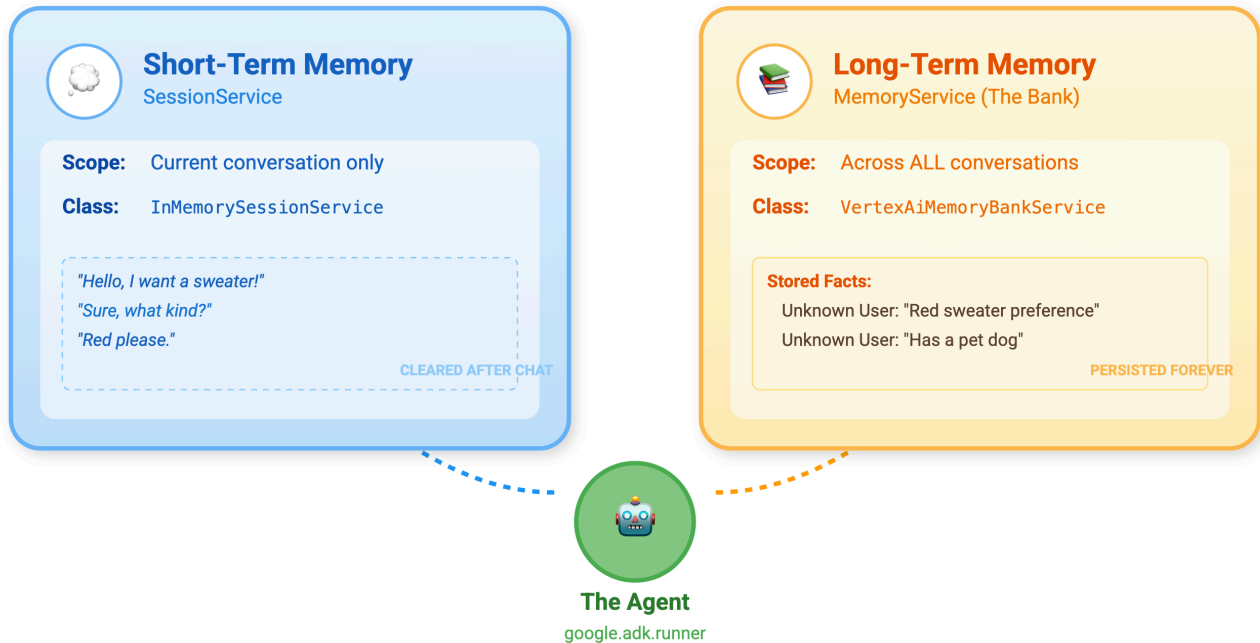
```

session_service = VertexAiSessionService(
    project=PROJECT_ID, location=LOCATION, agent_engine_id=AGENT_ENGINE_ID
)
memory_service = VertexAiMemoryBankService(
    project=PROJECT_ID, location=LOCATION, agent_engine_id=AGENT_ENGINE_ID
)

```

Memory Architecture

Short-Term vs. Long-Term Memory in backend/main.py



3. 🖱️ 在指令中輸入下列內容：

```
cloudshell edit ~/holiday_workshop/04-Adding-Memory-Bank/01-
starter/backend/deploy_agent.py
```

這會在編輯器中開啟 `~/holiday_workshop/04-Adding-Memory-Bank/01-starter/backend/deploy_agent.py`。

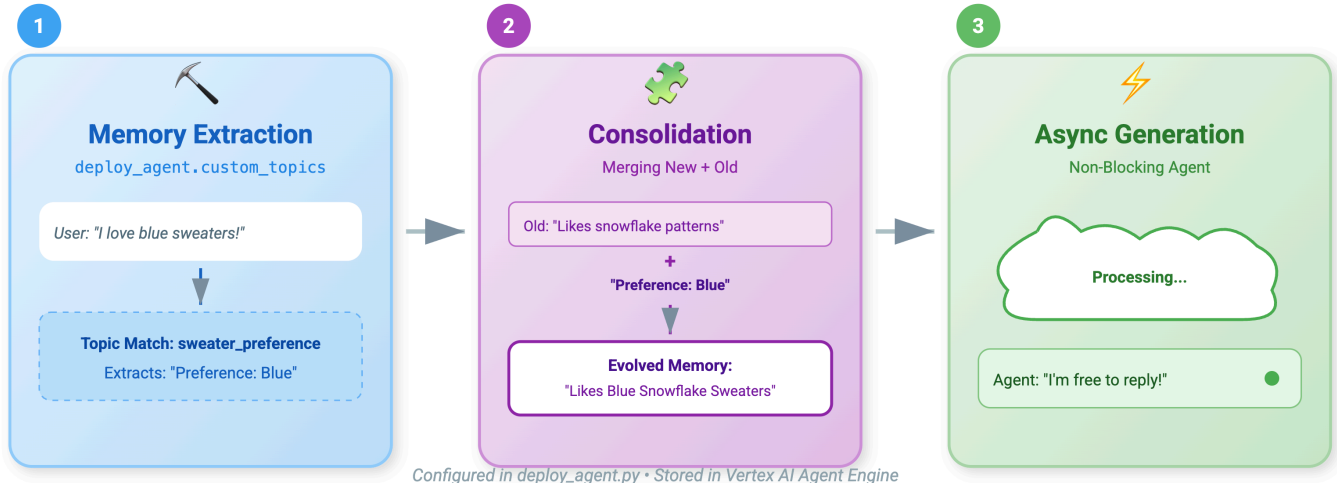
4. 將 `# TODO: Set Up Configuration` 替換為下列內容：


```
# Basic configuration types
MemoryBankConfig = types.ReasoningEngineContextSpecMemoryBankConfig
SimilaritySearchConfig = (
    types.ReasoningEngineContextSpecMemoryBankConfigSimilaritySearchConfig
)
GenerationConfig = types.ReasoningEngineContextSpecMemoryBankConfigGenerationConfig

# Advanced configuration types
CustomizationConfig = types.MemoryBankCustomizationConfig
MemoryTopic = types.MemoryBankCustomizationConfigMemoryTopic
CustomMemoryTopic = types.MemoryBankCustomizationConfigMemoryTopicCustomMemoryTopic
GenerateMemoriesExample = types.MemoryBankCustomizationConfigGenerateMemoriesExample
ConversationSource = (
    types.MemoryBankCustomizationConfigGenerateMemoriesExampleConversationSource
)
ConversationSourceEvent = (
    types.MemoryBankCustomizationConfigGenerateMemoriesExampleConversationSourceEvent
)
ExampleGeneratedMemory = (
    types.MemoryBankCustomizationConfigGenerateMemoriesExampleGeneratedMemory
)
```

Vertex AI Memory Bank Workflow

Automated Lifecycle of a Memory



核心功能

Vertex AI Memory Bank 不只是資料庫，更是主動式認知服務。

- **記憶體擷取**：系統會使用 LLM 讀取對話內容，並根據我們定義的 `CustomMemoryTopic` 結構定義 (例如「毛衣偏好」)，擷取有意義的事實。系統會忽略閒聊內容，並儲存訊號。
- **記憶體整合**：自動將新事實與舊事實合併。如果使用者先說「我喜歡紅色」，之後又說「其實我比較喜歡藍色」，Memory Bank 會演進記憶記錄，而不是建立衝突的重複項目。
- **非同步生成**：系統會在背景生成回憶集錦，記憶庫「思考」對話內容時，服務專員不會停滯不動。

5. 📌 在同一檔案中：04-Adding-Memory-Bank/01-starter/backend/deploy_agent.py。找出 # TODO: Set up topic，然後將整行替換為下列內容：

```
custom_topics = [
    # Topic 1: Sweater Preference
    MemoryTopic(
        custom_memory_topic=CustomMemoryTopic(
            label="sweater_preference",
            description="""Extract the user's preferences for sweater styles,
patterns, and designs. Include:
    - Specific patterns (snowflake, reindeer, geometric, fair isle, solid,
etc.)
    - Style preferences (chunky knit, cardigan, pullover, turtleneck,
oversized, fitted)
    - Color preferences (red, green, navy, pastel, etc.)
    - Material preferences if mentioned (wool, cotton, cashmere, itchy/soft)
    - Themes (retro, modern, ugly christmas sweater, elegant)

Example: "User wants a retro style sweater with a pixelated reindeer
pattern."

Example: "User prefers dark blue colors and hates itchy wool."
""",
        ),
    ),
    # Topic 2: Personal Context
    MemoryTopic(
        custom_memory_topic=CustomMemoryTopic(
            label="personal_context",
            description="""Extract the user's personal context including hobbies,
pets, interests, job, and preferred scenes. Include:
    - Hobbies and activities (skiing, reading, gaming, cooking, etc.)
    - Pets (type, breed, name, color)
    - Job or profession if relevant to their style
    - General interests (sci-fi, nature, vintage, tech)
    - Preferred scenes or vibes (cozy fireplace, snowy mountain, cyberpunk
city, beach)

Example: "User has a golden retriever named Max."
Example: "User loves skiing and wants a snowy mountain background."
Example: "User is a software engineer who likes cyberpunk aesthetics."
""",
        ),
    ),
]
```

6. 📌 在同一個檔案中：04-Adding-Memory-Bank/01-starter/backend/deploy_agent.py。找出 # TODO: Create Agent Engine，然後將整行替換為下列內容：

```

agent_engine = client.agent_engines.create(
    config={
        "display_name": AGENT_DISPLAY_NAME,
        "context_spec": {
            "memory_bank_config": {
                "generation_config": {
                    "model":
f"projects/{PROJECT_ID}/locations/{LOCATION}/publishers/google/models/gemini-2.5-flash"
                },
                "customization_configs": [customization_config]
            }
        },
    }
)

```

Agent Engine

Agent Engine 是雲端中的「大腦硬體」。管理：

- **向量資料庫**：用於儲存記憶。
- **建立索引**：方便搜尋回憶集錦。
- **API**：允許代理程式查詢。執行 `deploy_agent.py` 時，您基本上是在佈建這項硬體。

為什麼不直接使用提示？

您可能會問：「為什麼我們不直接將使用者的記錄貼到提示中？」

- **大小限制**：脈絡窗口很大，但並非無限大。無法顯示 5 年的記錄。
- **費用**：處理每個「Hello」的 100 萬個權杖，費用高得令人望之卻步。
- **重點**：Memory Bank 可做為代理程式的**搜尋引擎**。只會擷取**相關事實**。

7. 🖱️ 在指令中輸入下列內容：

```
cloudshell edit ~/holiday_workshop/04-Adding-Memory-Bank/01-starter/backend/agent.py
```

這會在編輯器中開啟 `~/holiday_workshop/04-Adding-Memory-Bank/01-starter/backend/agent.py`。

8. 在 `~/holiday_workshop/04-Adding-Memory-Bank/01-starter/backend/agent.py` 檔案中，將 `# TODO: Add PreloadMemoryTool` 替換為下列內容：

```

if USE_MEMORY_BANK:
    agent_tools.append(PreloadMemoryTool())

```

PreloadMemoryTool 和 **add_session_to_memory**

在 `agent.py` 中，您會看到兩個主要元件：

- **PreloadMemoryTool**：這項工具可讓服務專員「自行 Google」。如果使用者提出「幫我點一杯平常喝的咖啡」這類模糊要求，代理程式可以使用這項工具查詢 Memory Bank 中的「咖啡偏好設定」，再提供回覆。

- `add_session_to_memory`：這是背景回呼。
 - 為什麼要使用非同步？儲存記憶體需要時間 (總結對話、擷取事實)。我們不希望使用者等待這項作業完成。我們會使用 `after_agent_callback` 在背景 (`add_session_to_memory`) 執行這項作業。
-

步驟 10 · 約 0 分鐘

10. Memory Bank 實際應用

1. 🖱️ 在指令中輸入下列內容：

```
cd ~/holiday_workshop/04-Adding-Memory-Bank/01-starter
./use_memory_bank.sh
```

您會看到如下所示的結果：

```
INFO:httpx:HTTP Request: GET https://us-central1-aiplatform.googleapis.com/v1beta1/projects/738026075384/locations/us-central1/reasoningEngines/6475890391135551
s/3756526195496189952 "HTTP/1.1 200 OK"
INFO:vertexai_genai.agentengines:Agent Engine created. To use it in another session:
INFO:vertexai_genai.agentengines:agent_engine=client.agent_engines.get(name='projects/738026075384/locations/us-central1/reasoningEngines/6475890391135551488')
INFO: __main__:Agent Engine Registered Successfully!
INFO: __main__:Agent Engine ID: 6475890391135551488
INFO: __main__:
IMPORTANT: Add the following line to your backend/.env file:
INFO: __main__:AGENT_ENGINE_ID=6475890391135551488
Error: Could not extract Agent Engine ID from deployment output.
```

這個 `use_memory_bank.sh` 指令碼會先使用 `04-Adding-Memory-Bank/01-starter/backend/deploy_agent.py` 檔案將 Agent Engine 附加至代理，讓代理可以使用 **Vertex AI Memory Bank**。然後更新 `.env` 檔案，將代理程式引擎 ID 儲存為 `environmental variable`。

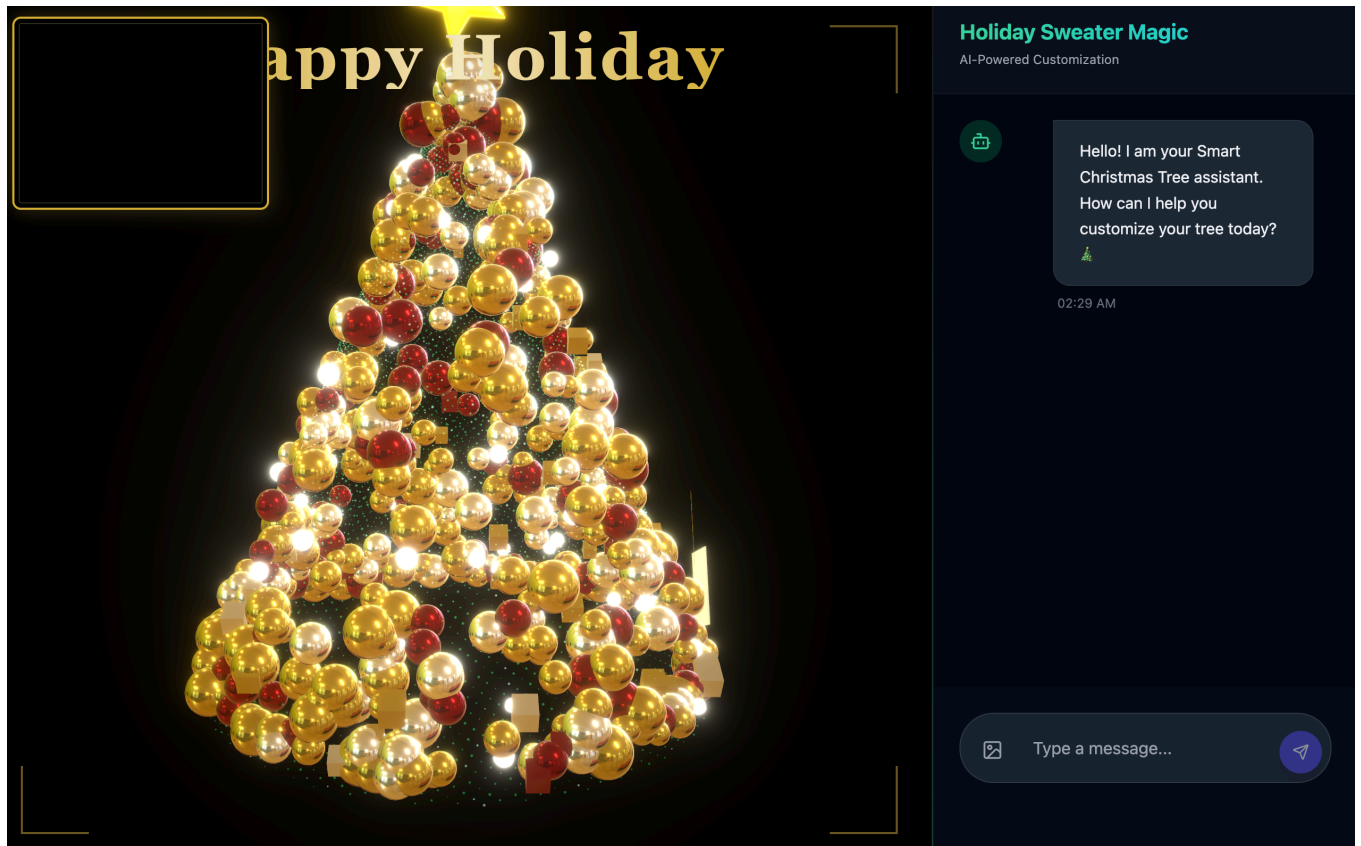
檢查 `~/holiday_workshop/.env` 檔案，您會看到 (不需採取任何動作)

```
USE_MEMORY_BANK=TRUE
AGENT_ENGINE_ID={agent_engine_id}
```

2. 🖱️ 使用應用程式 UI 測試記憶體。在指令中輸入下列內容：

```
cd ~/holiday_workshop/04-Adding-Memory-Bank/01-starter
./start_app.sh
```

請務必點選 `http://localhost:5173/`，或開啟新視窗並輸入 `http://localhost:5173/`。請注意，`Uvicorn` running on `http://0.0.0.0:8000` 只是後端伺服器，**不是**我們要點選的實際連結。現在，網站中的即時通訊介面已成為您的個人化代理程式！



3. 🖱️ 測試記憶體。如果您在使用者介面中輸入：

```
I want a sweater that matches my dog. He's a golden retriever.
```

```
I'm a programmer, so I want something geeky. Maybe a matrix style?
```

```
I like snowflake sweater pattern
```

代理程式會將此視為偏好設定，並儲存在 Memory Bank 中。

下週(或您隨時透過 `Control+C` 和 `./start_app.sh` 重新啟動應用程式時)，如果您提出以下要求：

```
what is my preference on sweater pattern?
```

代理程式會查詢**Memory Bank**，查看你的**偏好設定**，並在未收到要求的情況下生成毛衣圖案。

Holiday Sweater Magic

AI-Powered Customization



Hello! I am your Smart
Christmas Tree assistant.
How can I help you
customize your tree today?



02:54 AM

whats my preference on
sweater pattern?



02:55 AM



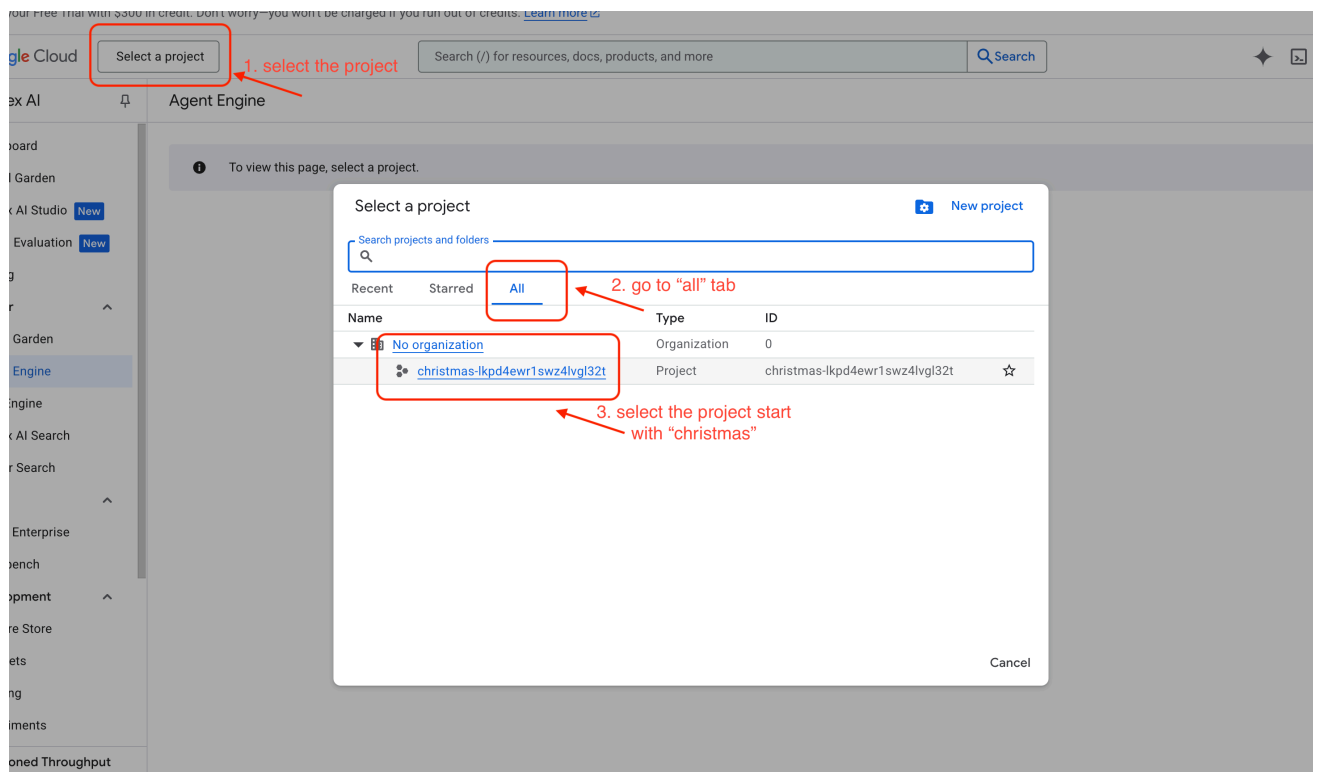
You mentioned you want a
sweater that matches your
golden retriever! So, your
preference is a ****golden
retriever pattern****! 🐾

Would you like me to generate a sweater pattern with a golden retriever motif, or perhaps a cute character wearing a sweater with that pattern? I can even

個人化：這是建構實用代理程式的關鍵。記住使用者偏好 (毛衣圖案、個人情境) 後，代理程式會隨著時間變得更有效率且實用。

4. 前往 [Google Cloud 控制台 Vertex AI Agent Engine](#)

- 請務必從左上方的專案選取器選取專案：



- 並驗證您剛從先前指令部署的代理程式引擎 `use_memory_bank.sh`

christmas-lkpd4ewr1swz4lvgl32t Search (/) for resources, docs, products, and more

Agent Engine

View and manage agent instances in this Cloud project.

[Develop agent](#) [Deployment guide](#)

Region: us-central1 (Iowa)

Filter Enter property name or value

Name	Resource name	Identity	Telemetry collection	Cr
christmas_tree_agent_engine_custom	.../6475890391135551488	...gserviceaccount.com	Learn more	De

點選您剛建立的代理程式引擎。

5. 點選已部署代理中的「**Memories**」分頁，即可查看所有記憶。

← christmas_tree_agent_engine_custom [Service configuration](#) [Copy query URL](#) [Co](#)

Dashboard Runtime metrics Traces Sessions Playground **Memories** Evaluation

Memory Bank stores long-term memories, which contain personalised information, to enable more context-aware agent interactions across multiple sessions. Here you can view, search and manage the agent's saved memories.

Memories [Refresh](#)

Filter Scope: key1:value1 key2:value2 ...

Scope	Resource name	Fact	Created	Updated
app_name : agents user_id : demo_user	.../9159362318176354304	I have a cat and want to ...	1 Dec 2025, 13:00:29	7 Dec 2025, 23:42:35
app_name : agents user_id : demo_user	.../5491180421683085312	I want a snowy mountai...	1 Dec 2025, 11:44:59	1 Dec 2025, 11:44:59
app_name : agents user_id : demo_user	...s/714778765321830400	I want a super cute sno...	28 Nov 2025, 00:40:26	7 Dec 2025, 23:49:46
app_name : agents user_id : demo_user	.../8208768545266335744	I have a golden retriever ...	28 Nov 2025, 00:40:26	1 Dec 2025, 11:31:19
app_name : agents user_id : demo_user	.../3597082526838947840	I love skiing.	28 Nov 2025, 00:40:26	1 Dec 2025, 11:31:19

恭喜！您已將記憶體庫附加到代理程式！

11. 結論

摘要

您已成功設計及建構完整的代理程式系統。

- **連線**：您使用 **MCP** 標準化代理程式存取本機工具的方式。
- **自動調度管理**：您使用 **ADK** 管理多步驟工作所需的複雜推論迴圈。
- **個人化**：您使用 **Memory Bank** 建立持續學習層，記住使用者情境。

後續步驟

- **建構自己的 MCP 伺服器**：為內部 API 或資料庫建立伺服器。
- **探索 ADK 模式**：瞭解 ADK 說明文件中的「推理迴圈」和「協調」。
- **部署**：將代理程式從本機指令碼移至 Cloud Run 的正式版服務。

持續建構：結合 MCP 的連線功能和 Memory Bank 的個人化功能，即可為新一代智慧型應用程式奠定基礎。